CHAPTER 2

NUMBER SYSTEMS, OPERATIONS, AND CODES

数字系统、运算和编码

2-1 DECIMAL NUMBERS

十进制数

Decimal（十进制） Review

- Numbers(数) consist of a bunch of digits（数字）, each with a weight（权重）.

1	6	2	.	3	7	5	Digits
100	10	1		1/10	1/100	1/1000	Weights

- These weights are all powers of the base, which is 10. We can rewrite this:

1	6	2	.	3	7	5	Digits
10^2	10^1	10^0		10^{-1}	10^{-2}	10^{-3}	Weights

- To find the decimal value of a number, multiply each digit by its weight and sum the products.

$$1\times10^2+6\times10^1+2\times10^0+3\times10^{-1}+7\times10^{-2}+5\times10^{-3}=162.375$$

$$D=\sum k_i*10^i$$

Nothing Special about 10!

- Decimal system (and the idea of "0") was invented in India around 100-500AD.
- Why did they use 10? Anything special about it?
  - Not really.
  - Probably the fact that we have 10 fingers influenced this.
- Will a base other than 10 work?
  - Sure.

$$345 \text{ in base } 9 = 3\times9^2 + 4\times9^1 + 5\times9^0 = 284 \text{ in base } 10$$

- What about base 2?

$$D = \sum k_i * N^i$$

2-2 BINARY NUMBERS

二进制数

Introductory Paragraph

- The binary number system is simply another way to represent quantities.
- A binary number is a weighted number（加权数）.
- The decimal system with its ten digits is a base-ten system; the binary system with its two digits is a base-two system. The two digits (bits) are 1 and 0. The position of a 1 or 0 in a binary number indicates its weight, or value within the number, just as the position of a decimal digit determines the value of that digit. The weights in a binary number are based on powers of two.

Introductory Paragraph

- The binary system is **less complicate**d than the decimal system because it has only two digits. It may seem more difficult at first because it is unfamiliar to you.

The Weighting Structure of Binary Numbers

- The right-most bit is the LSB（最低有效位 least significant bit） in a binary whole number and has a weight of $2^0=1$. The weights increase from right to left by a power of two for each bit. The left-most bit is the MSB（最高有效位 most significant bit）.
- Fractional numbers（小数） can also be represented in binary by placing bits to the right of the binary point. The left-most bit is the MSB in a binary fractional number and has a weight of $2^{-1}=0.5$. The fractional weights decreases from left to right by a negative power of two for each bit.

TABLE 2–2

Binary weights.

Positive Powers of Two (Whole Numbers)									Negative Powers of Two (Fractional Number)					
2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}
256	128	64	32	16	8	4	2	1	1/2 0.5	1/4 0.25	1/8 0.125	1/16 0.625	1/32 0.03125	1/64 0.015625

Counting in Binary

- A binary count of 0 through 15 is shown below. As you will see, 4 bits are required to count from 0 to 15.

TABLE 2–1

Decimal Number	Binary Number			
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

Binary-to Decimal Conversion

- The decimal value of any binary number can be found by adding the weights of all bits that are 1 and discarding the weights of all bits that are 0.

EXAMPLE 2–3 Convert the binary whole number 1101101 to decimal.

Solution Determine the weight of each bit that is a 1, and then find the sum of the weights to get the decimal number.

Weight: 2^6 2^5 2^4 2^3 2^2 2^1 2^0
Binary number: 1 1 0 1 1 0 1

$$1101101 = 2^6 + 2^5 + 2^3 + 2^2 + 2^0$$
$$= 64 + 32 + 8 + 4 + 1 = \mathbf{109}$$

Related Problem Convert the binary number 10010001 to decimal.

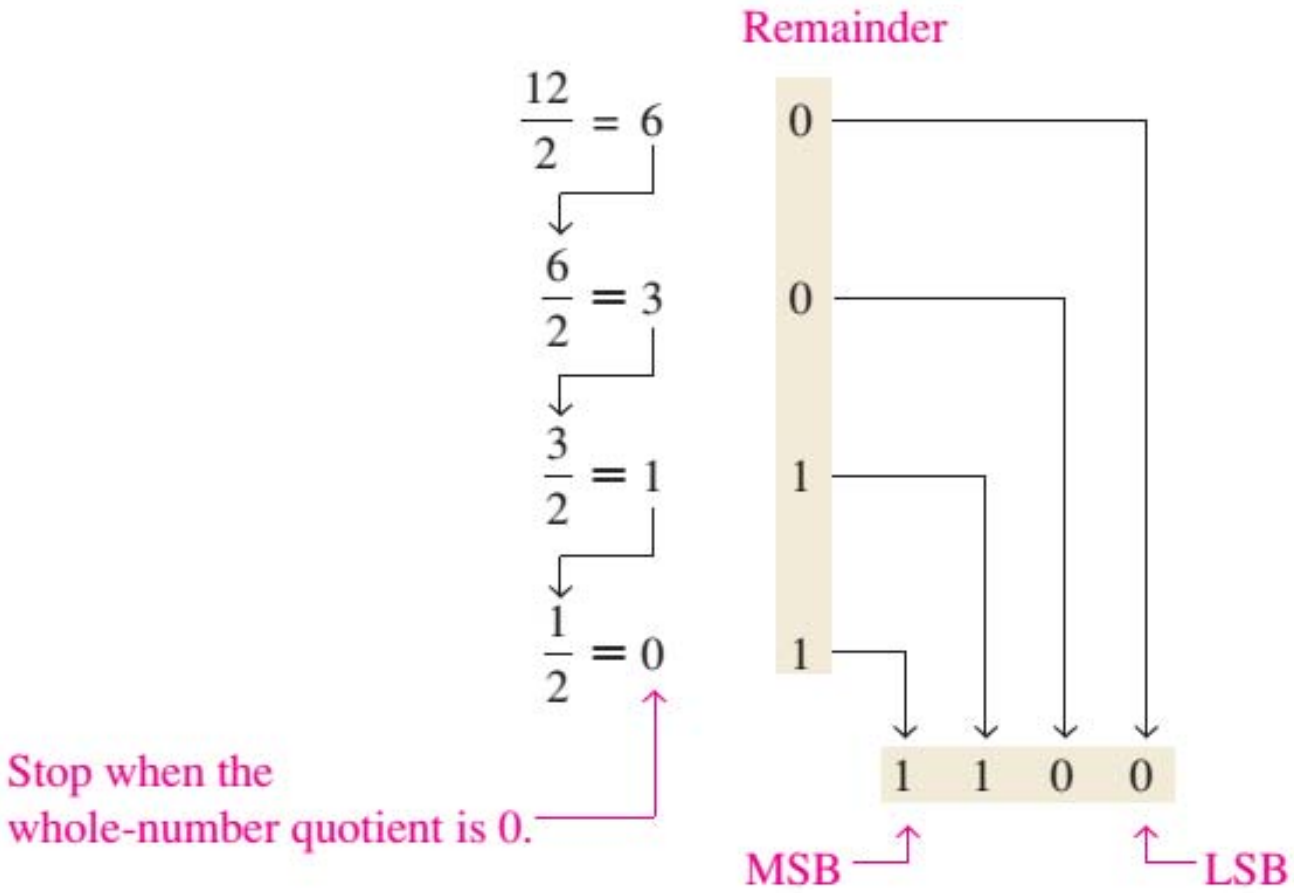
145

$$D = \sum k_i * N^i$$

2-3 DECIMAL-TO-BINARY CONVERSION

Repeated Division-by-2 Method

- A systematic method of converting whole numbers from decimal to binary is the repeated devision-by-2 process.

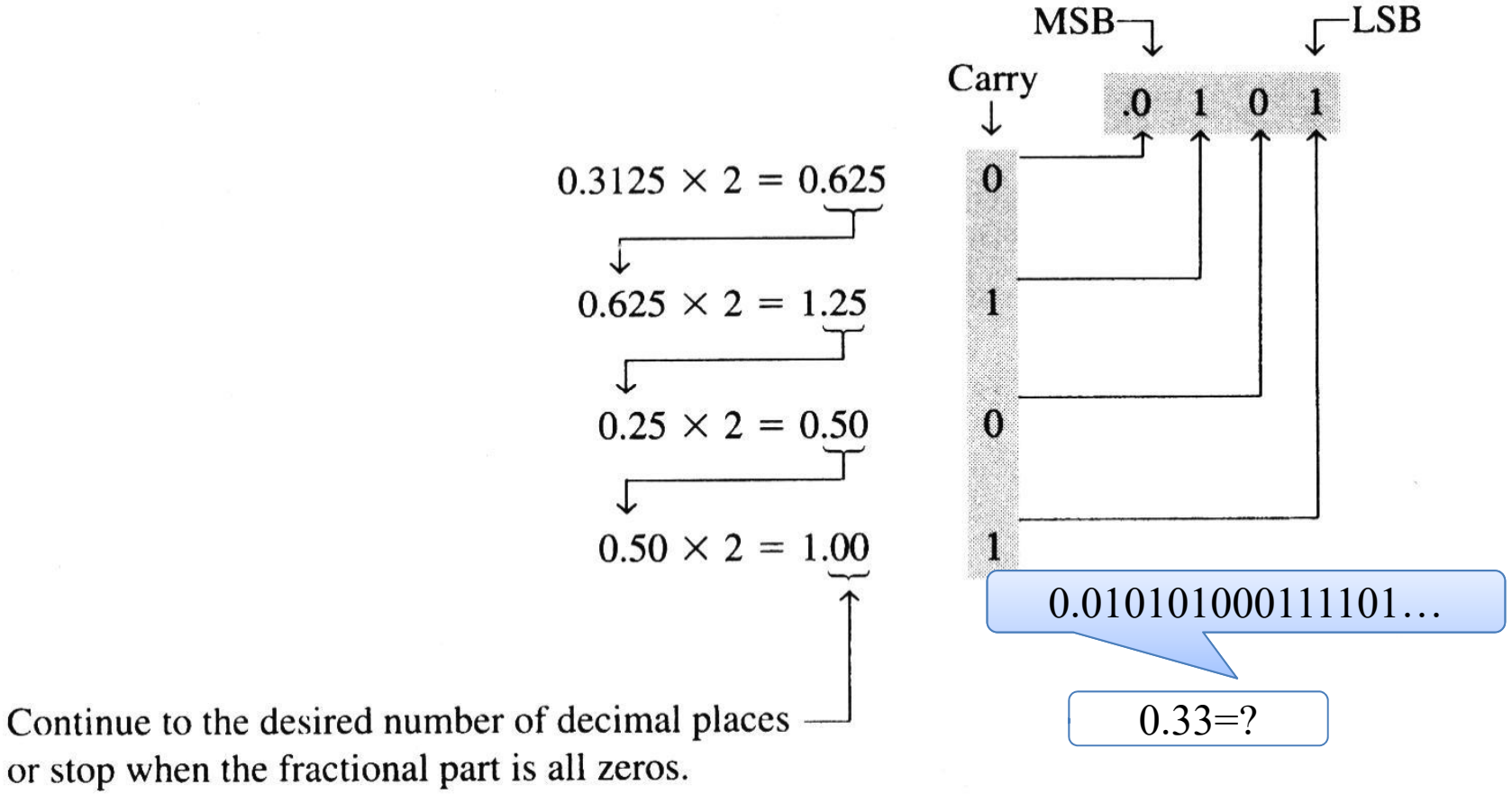


39?

10 0111

Repeated Multiplication-by-2 Method

- *A systematic method of converting fractional numbers from decimal to binary is the repeated multiplication-by-2 process.*



2-4 BINARY ARITHMETIC

Binary Addition

- The four basic rules for adding binary digits (bits) are as follows:

$0 + 0 = 0$ Sum of 0 with a carry of 0

$0 + 1 = 1$ Sum of 1 with a carry of 0

$1 + 0 = 1$ Sum of 1 with a carry of 0

$1 + 1 = 0$ Sum of 0 with a carry of 1

- [Example] Add 1111 and 1100.

11011

```
  1 1 1 1
+ 1 1 0 0
---------
1 1 0 1 1
```

Binary Subtraction

- The four basic rules for subtraction binary digits (bits) are as follows:

0 - 0 = 0 Difference of 0 with a borrow of 0

0 - 1 = 1 Difference of 1 with a borrow of 1

1 - 0 = 1 Difference of 1 with a borrow of 0

1 - 1 = 0 Difference of 0 with a borrow of 0

- [Example] Subtract 100 from 111.

```
    1 1 1
–   1 00
-------
      1 1
```

111-100=11

Binary Multiplication

- The four basic rules for multiplication binary digits (bits) are as follows:

$$0 \times 0 = 0$$

$$0 \times 1 = 0$$

$$1 \times 0 = 0$$

$$1 \times 1 = 1$$

- [Example] Multiply 1101 by 1010.

```
      1 1 0 1
  ×   1 0 1 0
-------------
      1 1 0 1
    1 1 0 1
-------------
1 0 0 0 0 0 1 0
```

Binary Division

- Division in binary follows the same procedure as division in decimal.

- [Example] Divide 1100 by 100.

```
          1 1
      ------
100 ) 1 1 0 0
        1 0 0
      -------
          1 0 0
          1 0 0
      -------
              0
```

2-5 SIGNED NUMBERS

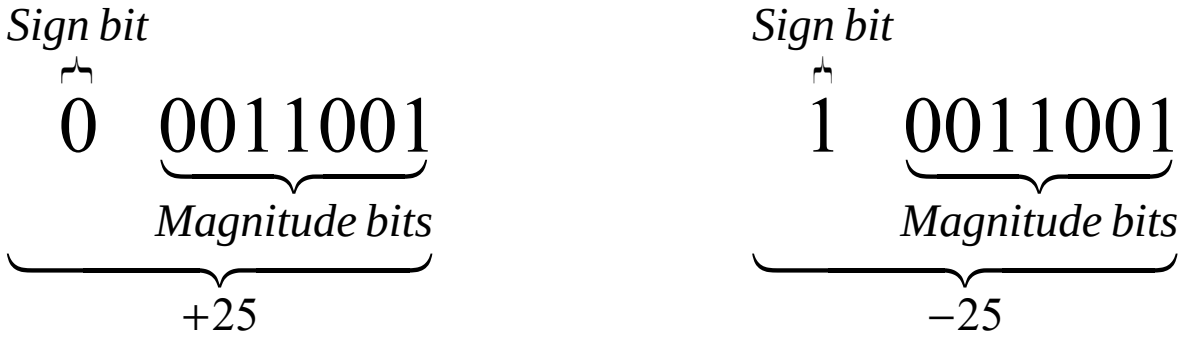
带符号数

The Sign Bit（符号位）

- The left-most bit in a signed binary number is the sign bit, which tells you whether the number is positive（正） or negative（负）. A 0 is for positive, and a 1 is for negative.

2.5.1 Sign-Magnitude(符号-幅值) System

- When a signed binary number is represented in sign-magnitude, the left-most bit is the sign bit and the remaining bits are the magnitude bits. The magnitude bits are in true (uncomplemented（非补码）) binary for both positive and negative numbers.



- The decimal values are determined by summing the weights in all the magnitude bit positions where there are 1s. The sign is determined by examination of the sign bit.

Sign-Magnitude System

- [Example 2-15] Determine the decimal value of this signed binary number expressed in sign-magnitude: 10010101.

Solution.

Summing the weights in all the magnitude bit positions where there are 1s,

$2^4 + 2^2 + 2^0 = 21$

The sign bit is 1; therefore, the decimal number is - 21.

2.5.2 The Complement System of Binary Numbers

1's complement and
2's complement

Finding the 1's Complement of a Binary Number

- The 1's complement (补码/反码) of a binary number is found by changing all 1s to 0s and all 0s to 1s.

1 0 1 1 0 0 1 0 Binary number

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓

0 1 0 0 1 1 0 1 1's complement

The simplest way to obtain the 1's complement of a binary number with a digital circuit is to use parallel inverters (NOT circuits), as shown in Figure 2–2 for an 8-bit binary number.

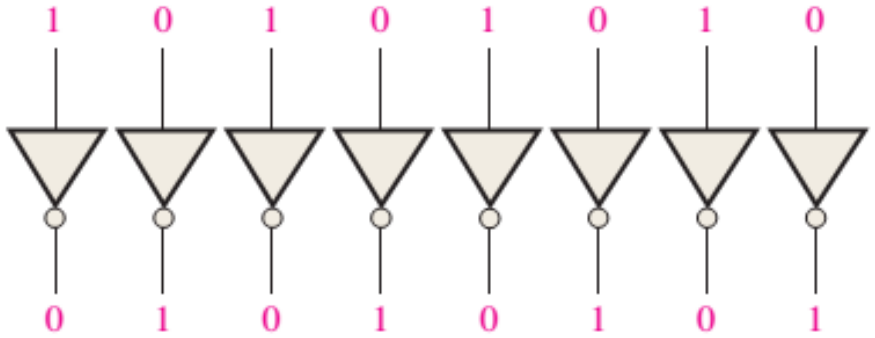


FIGURE 2–2 Example of inverters used to obtain the 1's complement of a binary number.

Definition of 1's Complement number

- **Positive numbers** in the 1's complement system are represented the same way as the positive sign-magnitude numbers.
- **Negative numbers**, however, are the 1's complements of the corresponding positive numbers.

$$\underbrace{00011001}_{+25} \qquad \underbrace{11100110}_{-25}$$

Conversion from a 1's Complement number to decimal

- The decimal values of positive numbers are determined by summing the weights in all bit positions where there are 1s.
- The decimal values of negative numbers are determined by summing the weights in all bit positions where there are 1s, and adding 1 to the result. The weight of the sign bit is given a negative value.

1's Complement System

- [Example 2-16] Determine the decimal value of the signed binary numbers expressed in 1's complement: (a) 00010111 (b) 11101000.

Solution.

(a) Summing the weights in all the magnitude bit positions where there are 1s,

$$2^4 + 2^2 + 2^1 + 2^0 = +23$$

(b) Summing the weights in all the magnitude bit positions where there are 1s,

$$-2^7 + 2^6 + 2^5 + 2^3 = -24$$

Adding 1 to the result

$$-24 + 1 = -23$$

1's Complement System

- Why?

$$\underbrace{00011001}_{+25} \qquad \underbrace{11100110}_{-25}$$

$$1110\ 0110 = 11111111 - 00011001$$

$$= 1\ 0000\ 0000 - 1 - 00011001$$

$$-00011001 = 1110\ 0110 - 1\ 0000\ 0000 + 1$$

$$= 1000\ 0000 + 110\ 0110 - 1\ 0000\ 0000 + 1$$

$$= -1000\ 0000 + 110\ 0110 + 1$$

$$= -2^7 + 2^6 + 2^5 + 2^2 + 2^1 + 1$$

2's Complement of a Binary Number

- The 2's complement（补）of a binary number is found by adding 1 to the LSB of the 1's complement.

$$\text{2's complement} = (\text{1's complement}) + 1$$

EXAMPLE 2–12

Find the 2's complement of 10110010.

Solution

	10110010	Binary number
	01001101	1's complement
+	1	Add 1
	01001110	2's complement

2's Complement System

- **Positive numbers** in the 2's complement system are represented the same way as in sign-magnitude and 1's complement systems.
- **Negative numbers** are the 2's complements of the corresponding positive numbers.

$$\underbrace{00011001}_{+25} \qquad \underbrace{11100111}_{-25}$$

Conversion from a 2's Complement number to decimal

- The decimal values are determined by summing the weights in all bit positions where there are 1s. The weight of the sign bit is given a negative value.

2's Complement System

- [Example 2-17] Determine the decimal value of the signed binary numbers expressed in 2's complement: (a) 01010110 (b) 10101010.

Solution.

(a) Summing the weights in all the magnitude bit positions where there are 1s,

$$2^6 + 2^4 + 2^2 + 2^1 = +86$$

(b) Summing the weights in all the magnitude bit positions where there are 1s,

$$-2^7 + 2^5 + 2^3 + 2^1 = -86$$

The 2's complement of an n-bit number x is defined by

$$[x]_{2c} = \begin{cases} x, & x \geq 0 \\ 2^n + x, & x < 0 \end{cases}$$

For $n = 8$, we have

$$[0000\ 0000]_{2c} = 0000\ 0000$$

$$[0000\ 0001]_{2c} = 0000\ 0001$$

$$[0000\ 0010]_{2c} = 0000\ 0010$$

$$\vdots$$

$$[01111111]_{2c} = 01111111$$

and

$$[-0000\ 0001]_{2c} = 1\ 0000\ 0000 - 0000\ 0001$$
$$= (1 + 11111111) - 0000\ 0001$$
$$= (11111111 - 0000\ 0001) + 1$$
$$= 11111110 + 1$$
$$= 11111111$$
$$[-0000\ 0010]_{2c} = 1\ 0000\ 0000 - 0000\ 0010$$
$$= (1 + 11111111) - 0000\ 0010$$
$$= (11111111 - 0000\ 0010) + 1$$
$$= 11111101 + 1$$
$$= 11111110$$
$$\vdots$$

2's Complement Advantage

- To convert to decimal
  - The 2's complement system simply requires a summation of weights regardless of whether the number is positive or negative.

(在二进制转十进制的计算中，采用2的补码形式表示的数字，不管是正数还是负数仅需要一个统一的权重求和的计算方式就可以实现)

  - The sign-magnitude system requires two steps - sum the weights of the magnitude bits and examine the sign bit to determine if the number is positive or negative.

(而采用'符号-幅值'表示形式则需要两个步骤——（1）对幅值表示部分进行权重求和；（2）判断符号位是正还是负)

2's Complement Advantage

- The 1's complement system requires adding 1 to the summation of weights for negative numbers but not for positive numbers.

(1的补码形式表示的二进制数，对于负数需要在最后的结果进行加1，对于正数则不需要，因此无法实现计算算法的统一)

- Also, the 1's complement system is not used because two representations of zero (00000000 or 11111111) are possible.

(1的补码形式另一个缺陷，零的表示有两种表示（00000000 or 11111111），表达形式不唯一)

Range of Signed Integer Numbers

- The number of different combinations of n bits is

$$Total\ combinations = 2^n$$

- For 2's complement signed numbers, the range of value for n-bit numbers is

$$-2^{n-1} \quad to \quad (2^{n-1} - 1)$$

- For 1's complement signed numbers, the range of value for n-bit numbers is

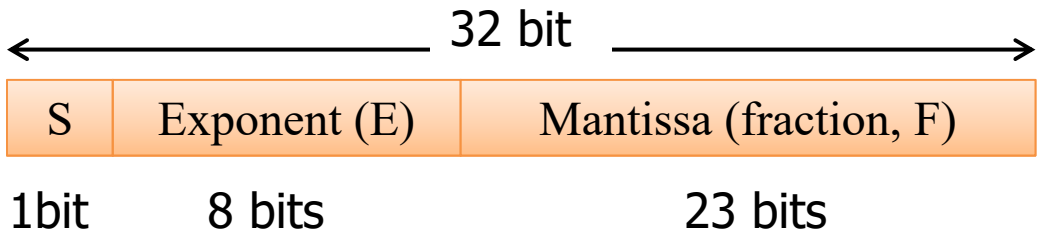
$$-2^{n-1} + 1 \quad to \quad (2^{n-1} - 1)$$

Floating-Point Numbers（浮点数）

- To represent very large numbers, many bits are required.
- The floating-point number, based on scientific notation, is capable of representing very large or very small numbers without increasing the number of bits.
- A floating-point number consists of two parts:

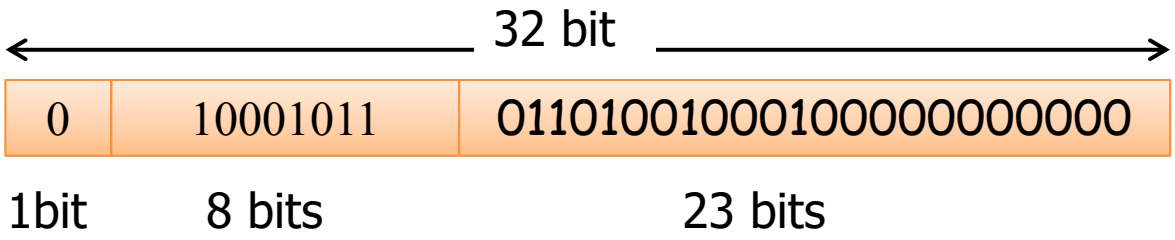
  Mantissa(定点尾数) and Exponent(指数)

Single-Precision Floating-Point Binary Number



Floating-Point Numbers（浮点数）

- Example: 1011010010001 (5777) = 1. 011010010001 x 2^12

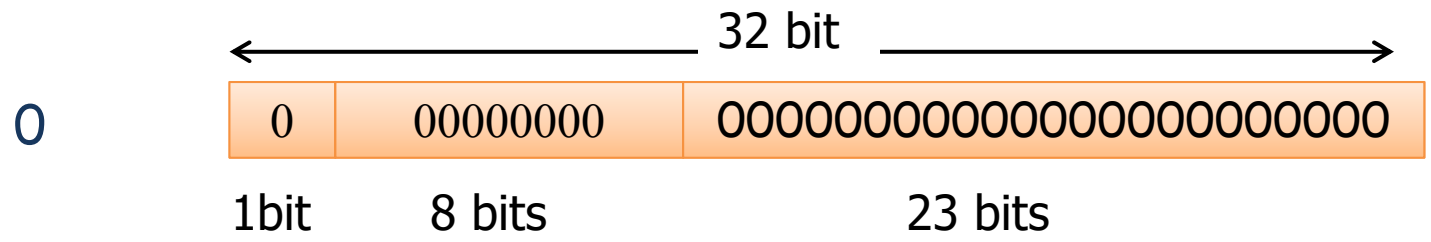


- Mantissa: the left-most bit '1' is understood to be there although it does not occupy an actual bit position. So effectively, there are 24 bits in the mantissa.

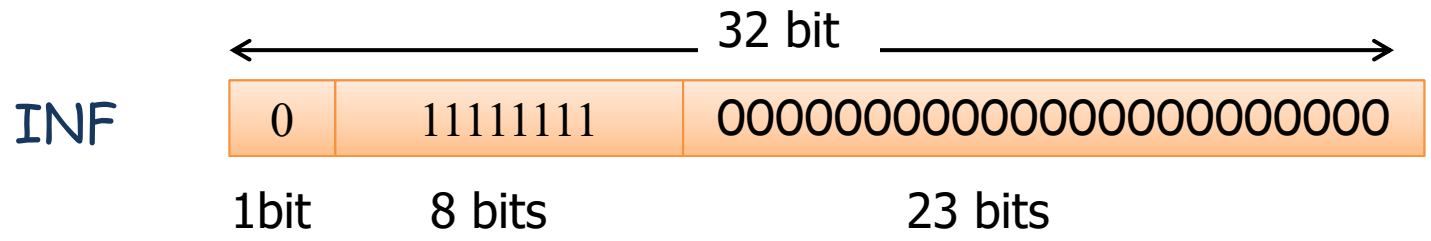
- Exponent(1~255): it is a biased exponent which is obtained by adding 127 to the actual exponent. So the range of an actual exponent values from -126 to +128.

Floating-Point Numbers（浮点数）

- 两个特殊数字：0.0 is represented by 0s



- Infinity(无穷大) is represented by all 1s in the exponent and all 0s in the mantissa.



- The 8-bit grouping has been given the special name byte.

Bit Grouping	Name
8-bit	Byte
16-bit	Word
32-bit	Double word / float number

2-6 ARITHMETIC OPERATIONS WITH SIGNED NUMBERS

带符号数的算术运算

Addition（加法）

- The two numbers in an addition are the augend（被加数） and the addend（加数）. The results are the sum （和）and the carry（进位）.

$$[x+y]_{2c} = [x]_{2c} + [y]_{2c}$$

- The addition process is stated as follows: **add the two numbers and discard any final carry bit.**

```
  00000111      7          00001111      15
+ 00000100    + 4        + 11111010    + −6
----------    ---        ----------    ----
  00001011     11          00001001      9
                          1
                          ↑
                     Discard carry
```

Overflow Condition（溢出条件）

- When two numbers are added and the number of bits required to represent the sum exceeds the number of bits in the two numbers, *an overflow results as indicated by an incorrect sign bit.*
- An overflow can occur only when both numbers are positive or both numbers are negative.

- [Example]
  (a) 01111101(125) + 00111010(58) =10110111(183) ?
  (b) 10001000 + 11101101 = ?

Subtraction（减法）

- The two numbers in a subtraction are the minuend（被减数） and the subtrahend（减数）. The results are the difference（差） and the borrow（借位）.

$$[x-y]_{2c}=[x+(-y)]_{2c}=[x]_{2c}+[-y]_{2c}$$

minuend　substrahend

- The subtraction process is stated as follows: **take the 2's complement of the subtrahend and add. Discard any final carry bit.**

- [Example]
  (a) 0001000 - 00000011 = ?
  (b) 11100111 - 00010011 = ?

EXAMPLE 2–20

Perform each of the following subtractions of the signed numbers:

(a) 00001000 − 00000011 **(b)** 00001100 − 11110111

Solution

Like in other examples, the equivalent decimal subtractions are given for reference.

(a) In this case, $8 - 3 = 8 + (-3) = 5$.

	00001000	Minuend (+8)
	+ 11111101	2's complement of subtrahend (−3)
Discard carry ⟶	**1 00000101**	Difference (+5)

(b) In this case, $12 - (-9) = 12 + 9 = 21$.

00001100	Minuend (+12)
+ 00001001	2's complement of subtrahend (+9)
00010101	Difference (+21)

Multiplication（乘法）

- The two numbers in a multiplication are the multiplicand（被乘数）and the multiplier（乘数）. The result is the product（积）.

- The multiplication operation in most computers is accomplished using partial product method（部分积方法）. The basic steps in the process are as follows:
  - Determine if the signs of the two numbers are the same. This determines what the sign of the product will be.
  - Change any negative number to **true (uncomplemented) form**.
  - Starting with the LSB of the multiplicand, generate the partial products. Shift each successive partial product one bit to the left.
  - Add each partial product to the sum of the previous partial products to get the final product.
  - If the sign of the product is negative, take the 2's complement of the product. Attach the sign bit to the product.

EXAMPLE 2–22

Multiply the signed binary numbers: 01010011 (multiplicand) and 11000101 (multiplier).

Solution

Step 1: The sign bit of the multiplicand is 0 and the sign bit of the multiplier is 1. The sign bit of the product will be 1 (negative).

Step 2: Take the 2's complement of the multiplier to put it in true form.

$$11000101 \longrightarrow 00111011$$

Step 3 and 4: The multiplication proceeds as follows. Notice that only the magnitude bits are used in these steps.

1010011	Multiplicand
× 0111011	Multiplier
1010011	1st partial product
+ 1010011	2nd partial product
11111001	Sum of 1st and 2nd
+ 0000000	3rd partial product
011111001	Sum

+ 1010011	4th partial product
1110010001	Sum
+ 1010011	5th partial product
100011000001	Sum
+ 1010011	6th partial product
1001100100001	Sum
+ 0000000	7th partial product
1001100100001	Final product

Step 5: Since the sign of the product is a 1 as determined in step 1, take the 2's complement of the product.

1001100100001 ⟶ 01100110111111

Attach the sign bit

1 01100110111111

Division （除法）

- The two numbers in a division are the dividend（被除数） and the divisor（除数）. The results are the quotient（商） and the remainder（余数）.

- The basic steps in a division the process are as follows:
  - Determine if the signs of the two numbers are the same. This determines what the sign of the quotient will be.
  - Change any negative number to **true (uncomplemented) form**.
  - Initialize the quotient to zero and initialize the partial remainder to the dividend.
  - Subtract the divisor from the partial remainder using 2's complement addition to get the next partial remainder. If the result is positive, add 1 to the quotient and repeat for the next partial remainder; otherwise, the division is complete.

EXAMPLE 2–23

Divide 01100100 by 00011001.

Solution

Step 1: The signs of both numbers are positive, so the quotient will be positive. The quotient is initially zero: 00000000.

Step 2: Subtract the divisor from the dividend using 2's complement addition (remember that final carries are discarded).

01100100	Dividend
+ 11100111	2's complement of divisor
01001011	Positive 1st partial remainder

Add 1 to quotient: 00000000 + 00000001 = 00000001.

Step 3: Subtract the divisor from the 1st partial remainder using 2's complement addition.

01001011	1st partial remainder
+ 11100111	2's complement of divisor
00110010	Positive 2nd partial remainder

Add 1 to quotient: 00000001 + 00000001 = 00000010.

Step 4: Subtract the divisor from the 2nd partial remainder using 2's complement addition.

00110010	2nd partial remainder
+ 11100111	2's complement of divisor
00011001	Positive 3rd partial remainder

Add 1 to quotient: 00000010 + 00000001 = 00000011.

Step 5: Subtract the divisor from the 3rd partial remainder using 2's complement addition.

00011001	3rd partial remainder
+ 11100111	2's complement of divisor
00000000	Zero remainder

Add 1 to quotient: 00000011 + 00000001 = **00000100** (final quotient). The process is complete.

Division

- [Example 2-23] Divide 01100100（100） by 00011001（25）
- [Example 2-23a] Divide 01100100（100） by 00011010（26）
- [Example 2-23b] Divide 01100100（100） by 11100111（-26）
- [Example 2-23c] Divide 10011100（-100） by 00011010 （+26）
- [Example 2-23d] Divide 10011100（-100） by 11100111 （-26）

2-8 HEXADECIMAL NUMBERS（十六进制数）

Why Hexadecimal?

- As you are probably aware, long binary numbers are difficult to read and write because it is easy to drop or transpose a bit. Since computers and microprocessors understand only 1s and 0s, it is necessary to use these digits when you program in machine language.

- 用二进制来表示一个数字需要占用较多的位数，书写起来较为麻烦，容易出错。但是计算机系统中只能识别0和1两个数字，因此在计算机的实际执行代码中必须采用二进制。

Why Hexadecimal?

- The hexadecimal number system has 16 digits and is used primarily as a compact way of displaying or writing binary numbers because it is very easy to convert between binary and hexadecimal.
- 十六进制数具有16个数字，是一种高效紧凑表示二进制数的一种技术方法，同时十六进制数和二进制数之间的转换也非常方便。
- 10 numeric digits (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) and 6 alphabetic characters (A, B, C, D, E, F) make up the hexadecimal number system.

Relationship between hexadecimal and binary

- Each hexadecimal digit represents a 4-bit binary number.

TABLE 2–3

Decimal	Binary	Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Binary-to-Hexadecimal Conversion

- Very straightforward! Simply break the binary number into 4-bit groups, starting at the right-most bit and replace each 4-bit group with the equivalent hexadecimal symbol.

EXAMPLE 2–24

Convert the following binary numbers to hexadecimal:

(a) 1100101001010111 **(b)** 1111110001011010 01

Solution

(a) 1100 1010 0101 0111

↓ ↓ ↓ ↓

C A 5 7 $= \mathbf{CA57}_{16}$

(b) 0011 1111 0001 0110 1001

↓ ↓ ↓ ↓ ↓

3 F 1 6 9 $= \mathbf{3F169}_{16}$

Two zeros have been added in part (b) to complete a 4-bit group at the left.

Hexadecimal-to-Binary Conversion

- Very straightforward! Simply replace each hexadecimal symbol with the equivalent 4-bit group.

EXAMPLE 2–25

Determine the binary numbers for the following hexadecimal numbers:

(a) $10A4_{16}$ **(b)** $CF8E_{16}$ **(c)** 9742_{16}

Solution

(a) 1 0 A 4

1 0000 1010 0100

(b) C F 8 E

1100 1111 1000 1110

(c) 9 7 4 2

1001 0111 0100 0010

In part (a), the MSB is understood to have three zeros preceding it, thus forming a 4-bit group.

Hexadecimal-to-Decimal Conversion

- Multiply the decimal value of each hexadecimal digit by its weight and then take the sum of these products.

EXAMPLE 2–27

Convert the following hexadecimal numbers to decimal:

(a) $E5_{16}$ **(b)** $B2F8_{16}$

Solution

Recall from Table 2–3 that letters A through F represent decimal numbers 10 through 15, respectively.

(a) $E5_{16} = (E \times 16) + (5 \times 1) = (14 \times 16) + (5 \times 1) = 224 + 5 = \mathbf{229}_{10}$

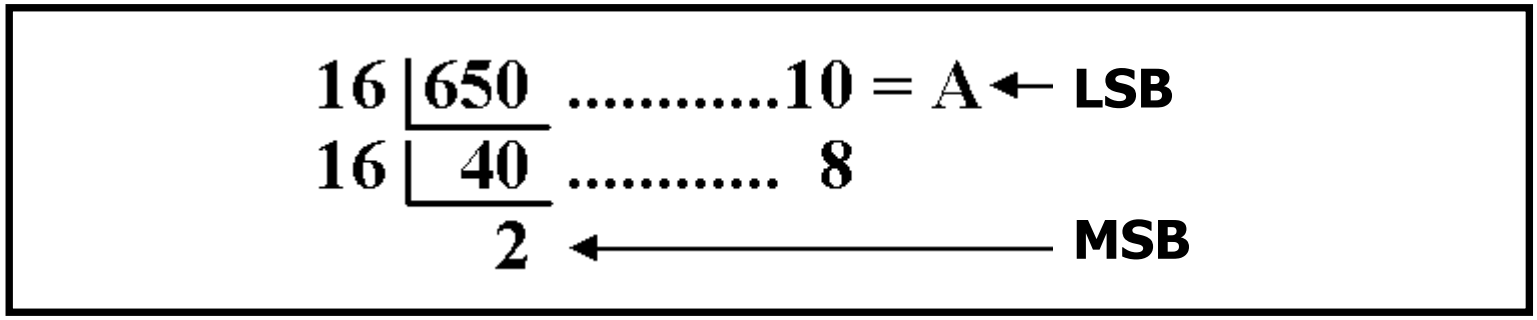
(b) $B2F8_{16} = (B \times 4096) + (2 \times 256) + (F \times 16) + (8 \times 1)$

$= (11 \times 4096) + (2 \times 256) + (15 \times 16) + (8 \times 1)$

$= 45{,}056 + 512 + 240 + 8 = \mathbf{45{,}816}_{10}$

Decimal-to-Hexadecimal Conversion

- Repeated division of a decimal number by 16 will produce the equivalent hexadecimal number.
- [Example 2-28] Convert the decimal number 650 to hexadecimal by repeated division by 16.



2-9 BINARY CODED DECIMAL (BCD)

Introductory Paragraph

- Binary coded decimal (BCD) is a way to express each of the decimal digits with a binary code. Since there are only ten code groups in the BCD system, it is very easy to convert between decimal and BCD. Because we like to read and write in decimal, the BCD code provides an excellent interface to binary systems. Examples of such interfaces are keypad inputs and digital readouts.

The 8421 Code

- The 8421 code is a type of BCD code.
- BCD means that each decimal digit, 0 through 9, is represented by a binary code of four bits.

TABLE 2–5
Decimal/BCD conversion.

Decimal Digit	0	1	2	3	4	5	6	7	8	9
BCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

- The designation 8421 indicates the binary weights of the four bits.
- 1010, 1011, 1100, 1101, 1110, and 1111 are invalid codes.
- The 8421 code is the predominant BCD code, and when we refer to BCD, we always mean the 8421 code unless otherwise stated.

Decimal	Binary	8421 BCD
0	0 0 0 0	0 0 0 0
1	0 0 0 1	0 0 0 1
⋮	⋮	⋮
9	1 0 0 1	1 0 0 1
10	1 0 1 0	0 0 0 1 0 0 0 0
11	1 0 1 1	0 0 0 1 0 0 0 1
12	1 1 0 0	0 0 0 1 0 0 1 0
13	1 1 0 1	0 0 0 1 0 0 1 1
14	1 1 1 0	0 0 0 1 0 1 0 0
15	1 1 1 1	0 0 0 1 0 1 0 1
16	1 0 0 0 0	0 0 0 1 0 1 1 0
17	1 0 0 0 1	0 0 0 1 0 1 1 1
18	1 0 0 1 0	0 0 0 1 1 0 0 0

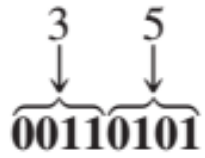
EXAMPLE 2–33

Convert each of the following decimal numbers to BCD:

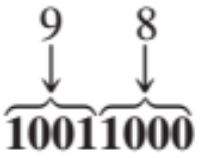
(a) 35 **(b)** 98

Solution

(a)



(b)



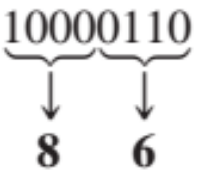
EXAMPLE 2–34

Convert each of the following BCD codes to decimal:

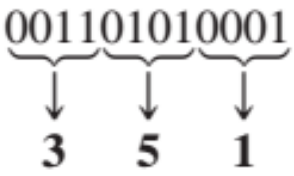
(a) 10000110 **(b)** 001101010001

Solution

(a)



(b)



BCD Addition

- BCD is a numerical code and can be used in arithmetic operations. Here is how to add two BCD numbers:
  - Add the two BCD numbers, using the rules for binary addition.
  - If a 4-bit sum is equal to or less than 9, it is a valid BCD number.
  - If a 4-bit sum is greater than 9, or if a carry out of the 4-bit group is generated, it is an invalid result. Add 6 (0110) to the 4-bit sum in order to skip the six invalid codes and returned the code to 8421. If a carry results when 6 is added, simply add the carry to the next 4-bit group.

EXAMPLE 2–35

Add the following BCD numbers:

(a) 0011 + 0100 **(b)** 00100011 + 00010101

Solution

The decimal number additions are shown for comparison.

(a)

0011	3
+ 0100	+ 4
0111	7

(b)

0010	0011	23
+ 0001	0101	+ 15
0011	**1000**	38

Note that in each case the sum in any 4-bit column does not exceed 9, and the results are valid BCD numbers.

EXAMPLE 2–36

Add the following BCD numbers:

(a) 1001 + 0100 **(c)** 00010110 + 00010101

Solution

The decimal number additions are shown for comparison.

(a)

1001			9
+ 0100			+4
1101		Invalid BCD number (>9)	13
+ 0110		Add 6	
0001	**0011**	Valid BCD number	
↓	↓		
1	3		

(c)

0001	0110		16
+ 0001	0101		+ 15
0010	1011	Right group is invalid (>9), left group is valid.	31
	+ 0110	Add 6 to invalid code. Add carry, 0001, to next group.	
0011	**0001**	Valid BCD number	
↓	↓		
3	1		

2-10 DIGITAL CODES AND PARITY
数字编码和奇偶校验

The Gray Code（格雷码）

- The Gray code is **unweighted** and is **not an arithmetic code**; that is, there are no specific weights assigned to the bit positions.
- The important feature of the Gray code is that it exhibits only a single bit change from one code number to the next.相邻码

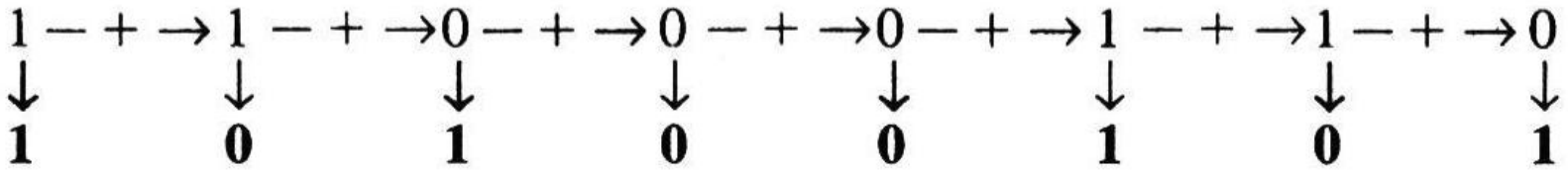
TABLE 2–6

Four-bit Gray code.

Decimal	Binary	Gray Code	Decimal	Binary	Gray Code
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

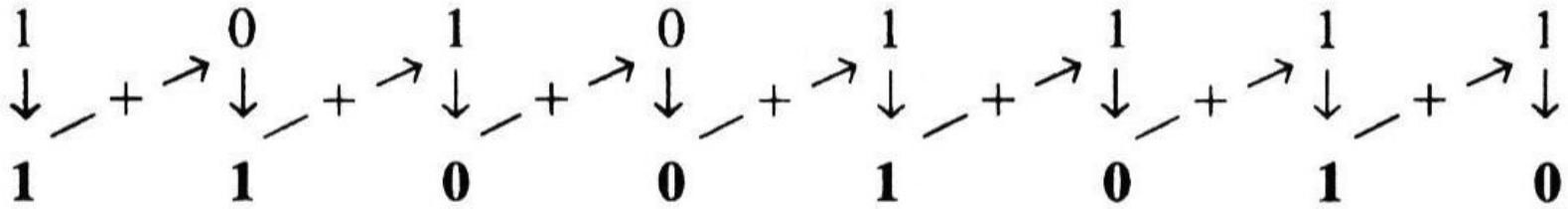
Binary-to-Gray code Conversion

- The MSB in the Gray code is the same as the corresponding MSB in the binary number.
- Going from left to right, add each adjacent pair of binary code bits to get the next Gray code bit. Discard carries.



Gray-to-Binary Conversion

- The MSB in the binary code is the same as the corresponding MSB in the Gray code.
- Going from left to right, add each binary code bit generated to the Gray code bit in the next adjacent position. Discard carries.



Application Example

- A simplified diagram of a 3-bit shaft position encoder mechanism（三位轴角位置编译器） is shown below.

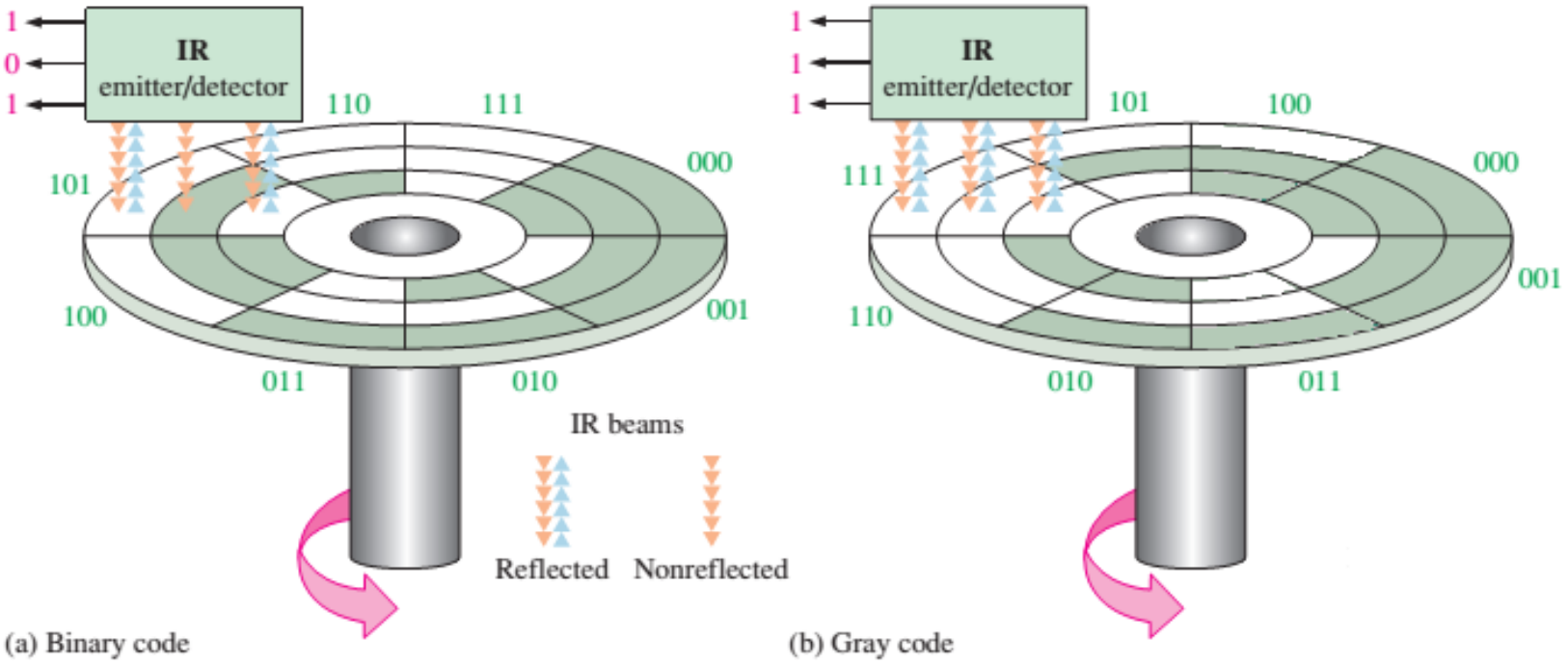
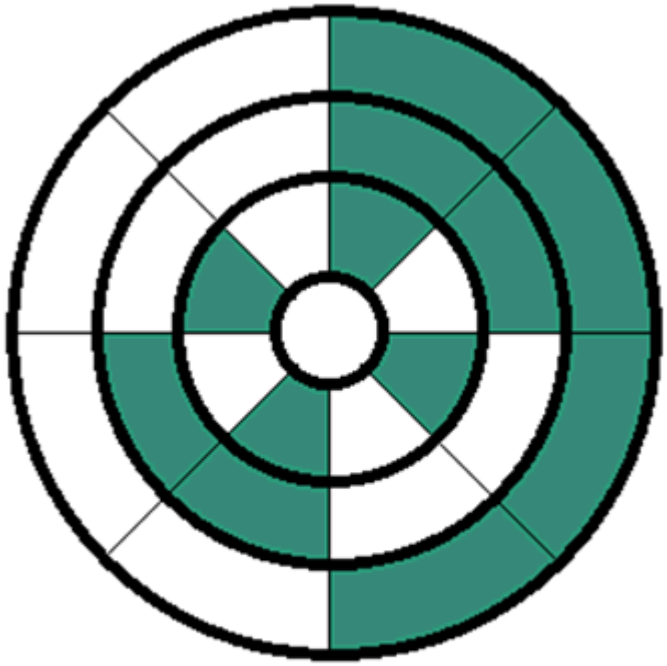
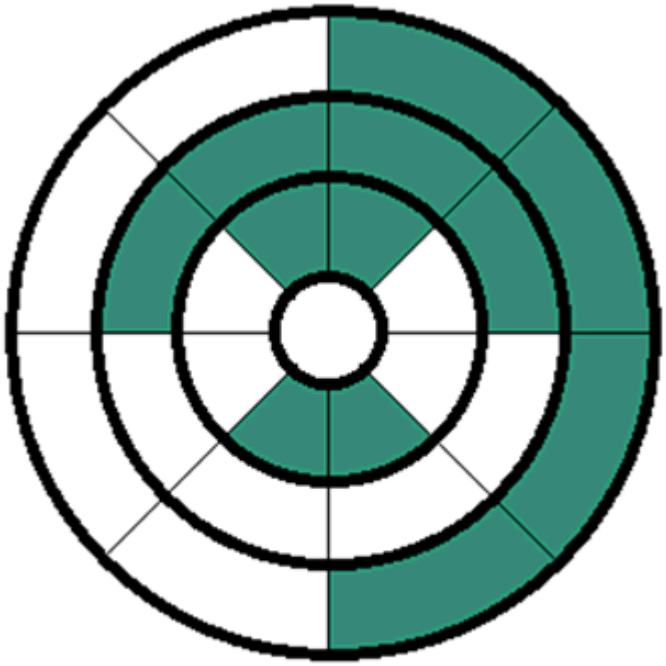


FIGURE 2–7 A simplified illustration of how the Gray code solves the error problem in shaft position encoders. Three bits are shown to illustrate the concept, although most shaft encoders use more than 10 bits to achieve a higher resolution.



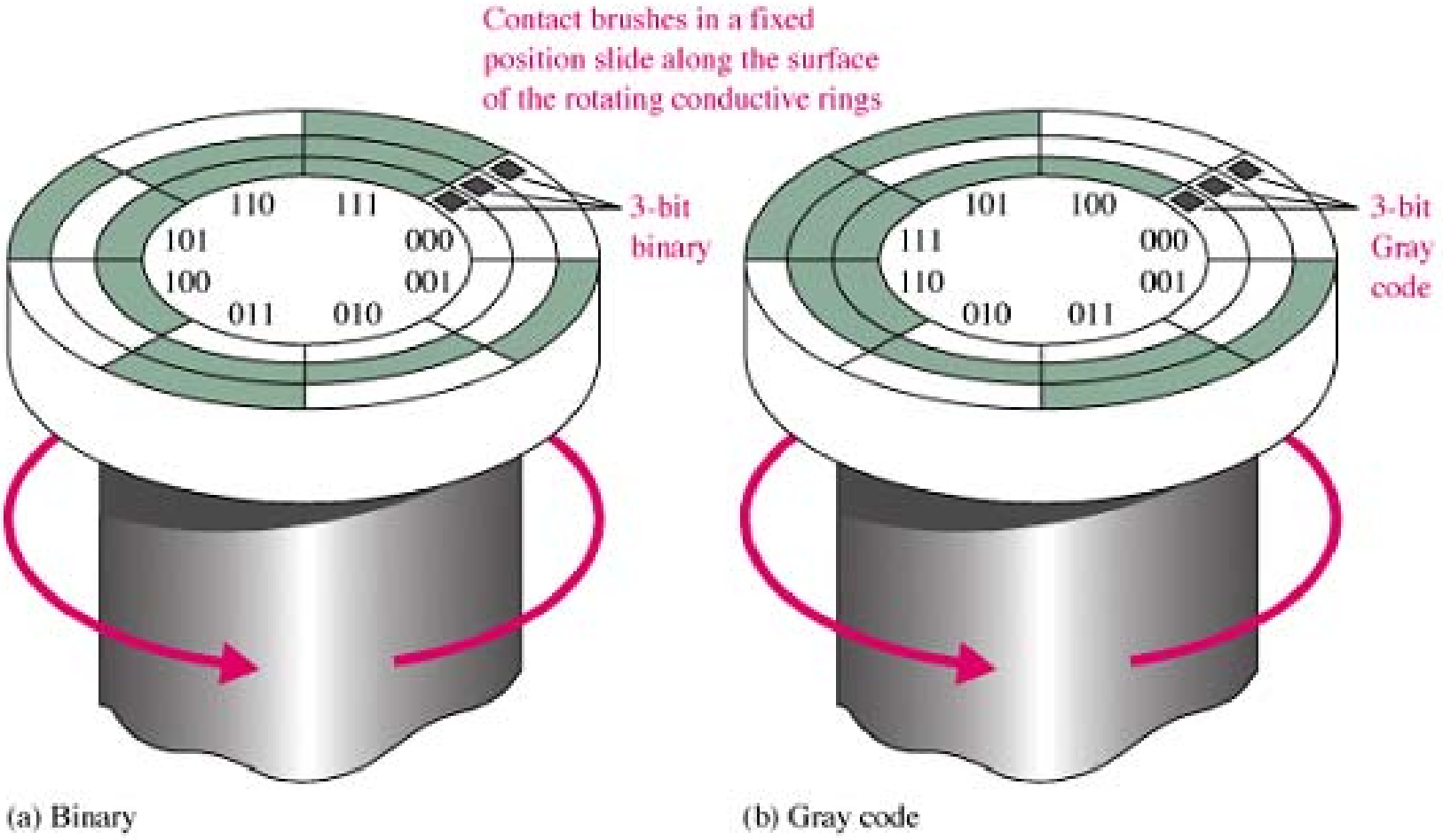
(a) Binary code



(b) Gray code

Application Example

- Consider what happens if one brush (for example, the MSB brush) is slightly ahead of the others during the transition from one sector to the next.



Alphanumeric Codes（字母数字编码）

- In the strictest sense, alphanumeric codes are codes that represent numbers and alphabetic characters (letters). Most such codes, however, also represent other characters such as symbols and various instructions necessary for conveying information. The ASCII is the most common alphanumeric code.
- ASCII has 128 characters represented by a 7-bit binary code.
  - The first 32 are nongraphic characters (control characters). That are never printed or displayed and used only for control purposes.
  - The others are graphic characters that can be printed or displayed and include the letters of the alphabet (lowercase and uppercase), the ten decimal digits, punctuation signs, and other commonly used symbols.

ASCII Code

十进制	十六进制	字符	十进制	十六进制	字符	十进制	十六进制	字符	十进制	十六进制	字符
0	0x00		32	0x20	[空格]	64	0x40	@	96	0x60	`
1	0x01		33	0x21	!	65	0x41	A	97	0x61	a
2	0x02		34	0x22	"	66	0x42	B	98	0x62	b
3	0x03		35	0x23	#	67	0x43	C	99	0x63	c
4	0x04		36	0x24	$	68	0x44	D	100	0x64	d
5	0x05		37	0x25	%	69	0x45	E	101	0x65	e
6	0x06		38	0x26	&	70	0x46	F	102	0x66	f
7	0x07		39	0x27	'	71	0x47	G	103	0x67	g
8	0x08	**	40	0x28	(	72	0x48	H	104	0x68	h
9	0x09	**	41	0x29	)	73	0x49	I	105	0x69	i
10	0x0A	**	42	0x2A	*	74	0x4A	J	106	0x6A	j
11	0x0B		43	0x2B	+	75	0x4B	K	107	0x6B	k
12	0x0C		44	0x2C	,	76	0x4C	L	108	0x6C	l
13	0x0D	**	45	0x2D	-	77	0x4D	M	109	0x6D	m
14	0x0E		46	0x2E	.	78	0x4E	N	110	0x6E	n
15	0x0F		47	0x2F	/	79	0x4F	O	111	0x6F	o
16	0x10		48	0x30	0	80	0x50	P	112	0x70	p
17	0x11		49	0x31	1	81	0x51	Q	113	0x71	q
18	0x12		50	0x32	2	82	0x52	R	114	0x72	r
19	0x13		51	0x33	3	83	0x53	S	115	0x73	s
20	0x14		52	0x34	4	84	0x54	T	116	0x74	t
21	0x15		53	0x35	5	85	0x55	U	117	0x75	u
22	0x16		54	0x36	6	86	0x56	V	118	0x76	v
23	0x17		55	0x37	7	87	0x57	W	119	0x77	w
24	0x18		56	0x38	8	88	0x58	X	120	0x78	x
25	0x19		57	0x39	9	89	0x59	Y	121	0x79	y
26	0x1A		58	0x3A	:	90	0x5A	Z	122	0x7A	z
27	0x1B		59	0x3B	;	91	0x5B	[	123	0x7B	{
28	0x1C		60	0x3C	<	92	0x5C	\	124	0x7C	|
29	0x1D		61	0x3D	=	93	0x5D	]	125	0x7D	}
30	0x1E		62	0x3E	>	94	0x5E	^	126	0x7E	~
31	0x1F		63	0x3F	?	95	0x5F	_	127	0x7F	

Parity Method for Error Detection

- Many systems use a parity bit as a means for bit error detection.
- Any group of bits contain either an even or an odd number of 1s. A parity bit is attached to a group of bits to make the total number of 1s in a group always even or always odd. An even/odd parity bit makes the total number of 1s even/odd.

As an illustration of how parity bits are attached to a code, Table 2–8 lists the parity bits for each BCD number for both even and odd parity. The parity bit for each BCD number is in the *P* column.

TABLE 2–8

The BCD code with parity bits.

Even Parity		Odd Parity	
P	**BCD**	***P***	**BCD**
0	0000	1	0000
1	0001	0	0001
1	0010	0	0010
0	0011	1	0011
1	0100	0	0100
0	0101	1	0101
0	0110	1	0110
1	0111	0	0111
1	1000	0	1000
0	1001	1	1001

EXAMPLE 2–39

Assign the proper even parity bit to the following code groups:

(a) 1010 **(b)** 111000 **(c)** 101101

(d) 1000111001001 **(e)** 101101011111

Solution

Make the parity bit either 1 or 0 as necessary to make the total number of 1s even. The parity bit will be the left-most bit (color).

(a) **0**1010 **(b)** **1**111000 **(c)** **0**101101

(d) **0**1000111001001 **(e)** **1**101101011111

Parity Method for Error Detection

- A parity bit provides for the detection of a single bit error (or any odd number of errors, which is very unlikely) but cannot check for two errors in one group.

EXAMPLE 2–40

An odd parity system receives the following code groups: 10110, 11010, 110011, 110101110100, and 1100010101010. Determine which groups, if any, are in error.

Solution

Since odd parity is required, any group with an even number of 1s is incorrect. The following groups are in error: **110011** and **1100010101010**.

Quiz

1. For the binary number 1000, the weight of the column with the 1 is

   a. 4

   b. 6

   c. 8

   d. 10

Quiz

2. The 2's complement of 1000 is

   a. 0111

   b. 1000

   c. 1001

   d. 1010

Quiz

3. The fractional binary number 0.11 has a decimal value of

   a. ¼

   b. ½

   c. ¾

   d. none of the above

Quiz

4. The hexadecimal number 2C has a decimal equivalent value of

   a. 14

   b. 44

   c. 64

   d. none of the above

Quiz

6. When two positive signed numbers are added, the result may be larger that the size of the original numbers, creating overflow. This condition is indicated by

   a. a change in the sign bit

   b. a carry out of the sign position

   c. a zero result

   d. smoke

Quiz

7. The number 1010 in BCD is

   a. equal to decimal eight

   b. equal to decimal ten

   c. equal to decimal twelve

   d. invalid

Quiz

8. An example of an unweighted code is

   a. binary

   b. decimal

   c. BCD

   d. Gray code

Quiz

9. An example of an alphanumeric code is

   a. hexadecimal

   b. ASCII

   c. BCD

   d. CRC

Quiz

10. An example of an error detection method for transmitted data is the

a. parity check

b. CRC

c. both of the above

d. none of the above

Homework

- Problems: 6(c,e,g), 7(b,e), 9(h), 13(h), 14(b)(保留小时后6位), 16(e,f), 17(d), 18(b), 22(h), 23(c)(d), 24(c)(d), 25(c)(d), 28(a)(b), 34(b), 37(g), 38(f), 39(g), 40(g), 42(b), 44(g), 47(k), 50(h), 52(f), 56(c), 57(c), 58.

Answers

1. c
2. b
3. c
4. b
5. a
6. a
7. d
8. d
9. b
10. c